

Carbon → Google

PYTHON

Fundamentals, Data Manipulation

Dart

Flutter

Author: Eng. Carlos Andrés Sierra, M.Sc.  
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor  
Computer Engineering Program  
School of Engineering  
Universidad Distrital Francisco José de Caldas

2026-I



UNIVERSIDAD DISTRITAL  
FRANCISCO JOSÉ DE CALDAS

# Outline

- 1 Foundations of Python
- 2 Numerical Analysis with Numpy
- 3 Data Manipulation with Pandas



# Outline

- 1 Foundations of Python
- 2 Numerical Analysis with Numpy
- 3 Data Manipulation with Pandas

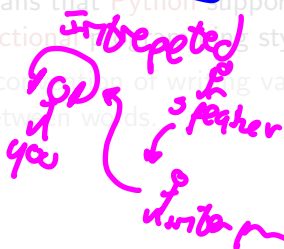


## Basic Definitions in Python

→ 1993 (4) → Guido Van Rossum

- **Python** is a **high-level**, **interpreted**, and **general-purpose** programming language.
- **High-level** means that **Python** is designed to be easy to read and write.

Mathematical Notation  
Natural Language



# Basic Definitions in Python

1st x = 5 / Java  
C

- **Python** is a **high-level**, **interpreted**, and **general-purpose** programming language.
- **High-level** means that **Python** is designed to be easy to read and write.
- **Interpreted** means that **Python** code is executed line by line, rather than being compiled into machine code.
- **Weakly typed** means that **Python** does not require you to declare the type of a variable.

- **Multiparadigm** means that **Python** supports **object-oriented**, **imperative**, and **functional** programming styles.
- **Snake-case** is the convention of writing variable names in **lowercase**, with **underscores** between words.

Python → C  
x = 9  
x = "hello world"  
x = False  
cpython

~ Py code  
↳ x.pyc



# Basic Definitions in Python

- **Python** is a **high-level**, **interpreted**, and **general-purpose** programming language.
- **High-level** means that **Python** is designed to be easy to read and write.
- **Interpreted** means that **Python** code is executed line by line, rather than being compiled into machine code.
- **Weakly typed** means that **Python** does not require you to declare the type of a variable.
- **Multiparadigm** means that **Python** supports **object-oriented**, **imperative**, and **functional** programming styles.
- **Snake-case** is the convention of writing variable names in **lowercase**, with **underscores** between words.



# Basic Definitions in Python

*Camel Case* } *my Variable Name*

- **Python** is a **high-level**, **interpreted**, and **general-purpose** programming language.
- **High-level** means that **Python** is designed to be easy to read and write.
- **Interpreted** means that **Python** code is executed line by line, rather than being compiled into machine code.
- **Weakly typed** means that **Python** does not require you to declare the type of a variable.
- **Multiparadigm** means that **Python** supports **object-oriented**, **imperative**, and **functional** programming styles.
- **Snake-case** is the convention of writing variable names in lowercase, with underscores between words.



*✓ my\_name\_variable*



# Popular Developer Tools

**Life is Short, Use Dev Tools** ByteByteGo

**Development Env**

- VS Code
- Visual Studio
- IntelliJ IDEA
- Notepad++
- Vim
- PyCharm
- Jupyter Notebook

**Diagramming**

- draw.io
- excalidraw
- mindmap
- Mermaid
- PlantUML
- Microsoft Visio
- Miro

**AI Tools**

- ChatGPT
- GitHub Copilot
- Tabnine
- Claude
- ollama
- Midjourney
- Stable Diffusion

**Hosting & Deployment**

- AWS
- Cloudflare
- Git Hub
- Fly.io
- Heroku
- Digital Ocean

**Code Quality**

- Jest
- ESLint
- Selenium
- sonarQube
- FindBugs
- Checkstyle

**Security**

- 1Password
- LastPass
- OWASP
- snyk
- Nmap

**Note-taking**

- Notion
- Markdown
- Obsidian
- Roam
- Logseq
- Tiddly Wiki

**Design**

- Figma
- Sketch
- Adobe Illustrator
- Canva
- Adobe Photoshop

**Handwritten Annotations:**

- Jet Brains** (circled around VS Code)
- PyTest** (circled around Jest)
- PyLint** (circled around ESLint)
- GCP** (circled around AWS)



# Virtual Environments

- **Virtual environments** are a way to create isolated spaces on your computer for **Python projects**.
- **Virtual environments** allow you to install packages and dependencies for a specific project without affecting other projects.

Python 2

Python 3

Python 2  
Numpy 1.4  
Keras 2.5

Python 3  
Numpy 2.4  
Keras 1.7



# Virtual Environments

- **Virtual environments** are a way to create isolated spaces on your computer for **Python projects**.
- **Virtual environments** allow you to **install packages** and **dependencies** for a specific project without affecting other projects.
- **Virtual environments** are created using the **venv** module, which is included in the Python standard library.
- **Virtual environments** are activated using the **source** command in the terminal.
- **Virtual environments** are deactivated using the **deactivate** command in the terminal.



# Virtual Environments

- **Virtual environments** are a way to create isolated spaces on your computer for **Python projects**.
- **Virtual environments** allow you to **install packages** and **dependencies** for a specific project without affecting other projects.
- **Virtual environments** are created using the **venv** module, which is included in the Python standard library.
- **Virtual environments** are activated using the **source** command in the terminal.
- **Virtual environments** are deactivated using the **deactivate** command in the terminal.



# Modules and Packages

- **Modules** are Python files that contain functions, classes, and variables.
- **Modules** are used to organize code and make it reusable.

• Packages are directories that contain Python files (modules).

• Packages are used to organize modules and make them reusable.

test.py

main.py



# Modules and Packages

- **Modules** are Python files that contain functions, classes, and variables.
- **Modules** are used to organize code and make it reusable.
- **Packages** are directories that contain Python files (modules).
- **Packages** are used to organize modules and make them reusable.

Handwritten notes and diagrams:

- A blue arrow points from the word "Packages" in the third bullet point to the handwritten text "Java".
- A blue arrow points from the word "modules" in the fourth bullet point to the handwritten text "--init--py".



# Dependencies Management

- **Dependencies** are external libraries or packages that your **Python project** relies on.
  - **Dependencies** are traditionally managed using a pyproject.toml file, which lists the names and versions of the packages required by your project.
  - **Dependencies** can be installed using the **pip** package manager, which is included with Python.
  - Currently, the **poetry** package manager is recommended for managing dependencies in Python projects. It follows the **PEP 508** standard for specifying dependencies.
- avoid conflicts



# Dependencies Management

- **Dependencies** are **external libraries** or **packages** that your **Python project** relies on.
- **Dependencies** are traditionally managed using a **pyproject.toml** file, which lists the **names** and **versions** of the packages required by your project.
- **Dependencies** can be installed using the **pip** package manager, which is included with Python.
- Currently, the **poetry** package manager is recommended for managing dependencies in Python projects. It follows the **PEP 508** standard for specifying dependencies.



# Jupyter Notebooks

## Data Analysis

- **Jupyter Notebooks** are a web-based interactive computing environment that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.

- Jupyter Notebooks are used for data cleaning, data transformation, statistical modeling, data visualization, machine learning, and more.
- Jupyter Notebooks support multiple programming languages, including Python, R, and Julia.
- Jupyter Notebooks are used by data scientists, researchers, students, and educators to explore data, create reports, and teach programming.

education { Carol Willing  
Fernando Pérez



# Jupyter Notebooks

- **Jupyter Notebooks** are a **web-based interactive computing environment** that allows you to create and share documents that contain **live code**, **equations**, **visualizations**, and **narrative text**.
- **Jupyter Notebooks** are used for data cleaning, data transformation, statistical modeling, data visualization, machine learning, and more.
- Jupyter Notebooks support multiple programming languages, including Python, R, and Julia.
- Jupyter Notebooks are used by data scientists, researchers, students, and educators to explore data, create reports, and teach programming.

basic

classic



# Jupyter Notebooks

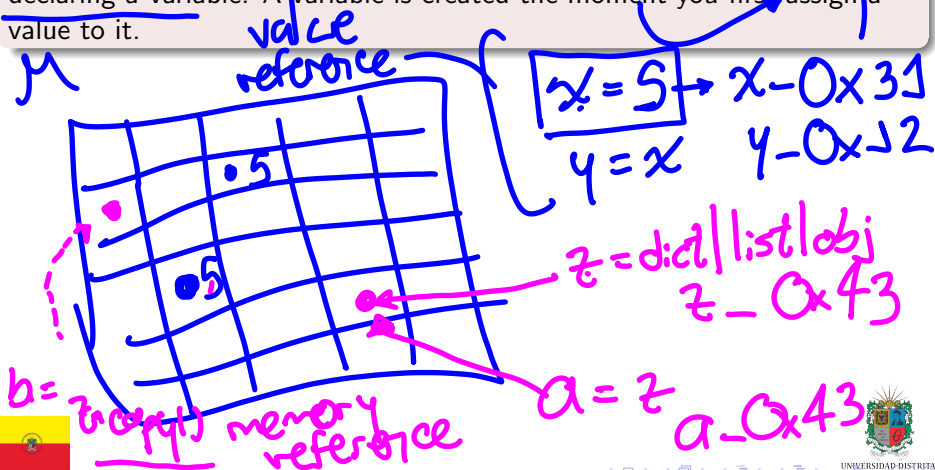
- **Jupyter Notebooks** are a **web-based interactive computing environment** that allows you to create and share documents that contain **live code**, **equations**, **visualizations**, and **narrative text**.
- **Jupyter Notebooks** are used for **data cleaning**, **data transformation**, **statistical modeling**, **data visualization**, **machine learning**, and **more**.
- **Jupyter Notebooks** support **multiple programming languages**, including **Python**, **R**, and **Julia**.
- **Jupyter Notebooks** are used by **data scientists**, **researchers**, **students**, and **educators** to **explore data**, **create reports**, and **teach programming**.



# Variables and Memory Management

## Variables Definition

**Variables** are used to store data values. Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.



# Conditionals

## Definition

**Conditionals** are used to execute different code blocks based on *different conditions*.

## Nested Conditionals

**Nested conditionals** are conditionals that are used with other conditionals.

## Elif Conditionals

**Elif conditionals** are used to execute multiple conditions.

if condition:

tab

else:

tab



# Conditionals

## Definition

**Conditionals** are used to execute different code blocks based on *different conditions*.

## Nested Conditionals

**Nested conditionals** are conditionals that are *inside* other conditionals.

Elif Conditionals

Elif conditionals are used to check *multiple conditions*.

*if cond\_1:*

*.... if cond\_2:*

*.... .... if cond\_3:*



# Conditionals

## Definition

**Conditionals** are used to execute different code blocks based on *different conditions*.

## Nested Conditionals

**Nested conditionals** are conditionals that are *inside* other conditionals.

## Elif Conditionals

**Elif conditionals** are used to check *multiple conditions*.

(switch)

else if  
elif

if cond-1:  
elif cond-2:  
elif cond-3:  
else:



# Loops and Range

~~(i=0; i<=n; i++)~~

## Loops Definition

**Loops** are used to execute a block of code *multiple times*.

## Range Definition

The **range function** is used to generate a sequence of numbers.

while

for

while cond:  
....

for item in set:  
....



# Loops and Range

## Loops Definition

**Loops** are used to execute a block of code *multiple times*.

## Range Definition

The **range** function is used to generate a sequence of numbers.

$\text{range}(\text{stop})$ :  $(0, 1, 2, \dots, \text{stop}-1)$

$\text{range}(\text{start}, \text{stop})$ :  $\text{start}, \text{start}+1, \dots, \text{stop}-1$

$\text{range}(\text{start}, \text{stop}, \text{int})$ :  
 $\text{start}, \text{start}+\text{int}, \dots, \text{stop}-1$

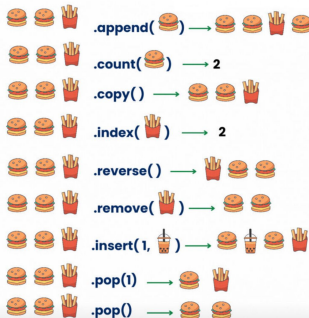


# Lists

## Definition

A **list** is a collection of items that are *ordered* and *changeable*. **Lists** are defined by enclosing the items in square brackets.

### PYTHON LIST METHODS



# Dictionaries

## Definition

A **dictionary** is a collection of items that are *unordered*, *changeable*, and *indexed*. Dictionaries are defined by enclosing the items in curly braces.



# Sets and Tuples

## Definition Sets

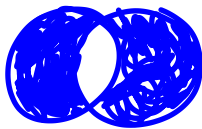
A set is a collection of items that are unordered and unindexed. **Sets** are defined by enclosing the items in curly braces.

## Definition Tuples

A tuple is a collection of items that are ordered and unchangeable. **Tuples** are defined by enclosing the items in parentheses.

$A = \{a, b, c, d, \dots, z\}$  <sup>unique</sup> set

$\{a:1, b:2, \dots\}$  dict



$A \cap B$



# Sets and Tuples

## Definition Sets

A **set** is a collection of items that are *unordered* and *unindexed*. **Sets** are defined by enclosing the items in curly braces.

## Definition Tuples

A **tuple** is a collection of items that are *ordered* and *unchangeable*. **Tuples** are defined by enclosing the items in parentheses.

$tuple = (a, b, c)$   
 $tuple = (a,)$



# Lists Comprehensions

## Definition

**List comprehensions** provide a concise way to create lists. Common applications are to make *new lists* where each element is the result of some operation applied to each member of another sequence or iterable, or to create a subsequence of those elements that satisfy a certain condition.

$[oper(i) \text{ for } i \text{ in range}(n)]$

$[oper(i_0), oper(i_1), \dots, oper(i_n)]$

$l = []$   
 for  $i$  in  $range(n)$ :  
 $l.append(oper(i))$



# Functions

Define f as a function...

## Definition

A **function** is a block of code that only runs when it is called. You can pass data, known as parameters, into a function. A **function** can return data as a result.

def f(args):

... ~

~

~

return..



# Functions

## Definition

A **function** is a block of code that only runs when it is called. You can pass data, known as parameters, into a function. A **function** can return data as a result.

## Type of Functions

- **Built-in** Functions → *native*
- **User-defined** Functions
  - Variadic Functions
  - **R**ecursive Functions



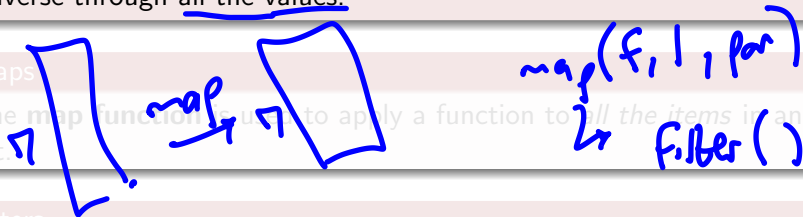
# Python Iterators

## Definition

An **iterator** is an object that contains a countable number of values. An **iterator** is an object that can be iterated upon, meaning that you can traverse through all the values.

## Maps

The **map** function is used to apply a function to *all the items* in an input list.



## Filters

The **filter** function is used to *select items* from an input list that meet a certain condition.



# Python Iterators

## Definition

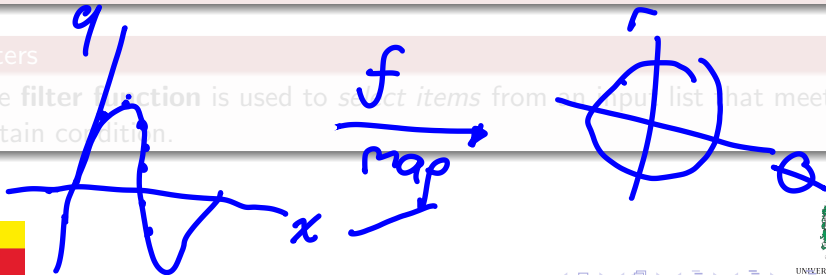
An **iterator** is an object that contains a *countable number of values*. An **iterator** is an object that can be iterated upon, meaning that you can traverse through all the values.

## Maps

The **map** function is used to apply a function to *all the items* in an input list.

## Filters

The **filter** function is used to select items from an input list that meet a certain condition.



# Python Iterators

## Definition

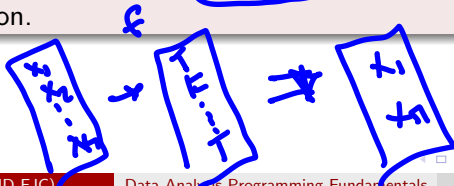
An **iterator** is an object that contains a *countable number of values*. An **iterator** is an object that can be iterated upon, meaning that you can traverse through all the values.

## Maps

The **map function** is used to apply a function to *all the items* in an input list.

## Filters

The **filter function** is used to *select items* from an input list that meet a certain condition.



# Lambda Functions

## Definition

A lambda function is a small anonymous function. A lambda function can take any number of arguments, but can only have one expression.

```
def sum(a,b):
    return a+b
```

$\lambda a,b: a+b$

```
def p3-to-str(x):
    return str(x**3)
```

$\lambda x: str(x**3)$

```
def get_book(book):
    return book['isbn']
```

$\lambda book: book['isbn']$



# Classes and Objects

## Definition

Python is an *object-oriented programming* language. Almost everything in Python is an **object**, with its *properties* and *methods*. A **class** is like an object constructor, or a “blueprint” for creating objects.



# Outline

- 1 Foundations of Python
- 2 Numerical Analysis with Numpy
- 3 Data Manipulation with Pandas



# Numerical Python Library — Numpy

- **Numpy** is the **core** library for **scientific computing** in Python. It is the **fundamental package** for scientific computing with **Python**.
- **Numpy** is a general-purpose **array-processing** package. It provides a **high-performance** multidimensional array object, and tools for working with these arrays.
- **Numpy** was created by **Travis Oliphant** in 2005, and it is an **open-source project**. Currently, **Numpy version 2.4** is available.



# Numerical Python Library — Numpy

- **Numpy** is the **core** library for **scientific computing** in Python. It is the **fundamental package** for scientific computing with **Python**.
- **Numpy** is a general-purpose **array-processing** package. It provides a **high-performance** multidimensional array object, and tools for working with these arrays.
- **Numpy** was created by **Travis Oliphant** in 2005, and it is an **open-source project**. Currently, **Numpy version 2.4** is available.



# Lineal Algebra with Numpy

- **Numpy** provides a comprehensive set of linear algebra functions
- **Numpy** provides the functionality to create and manipulate matrices.
- Numpy provides the functionality to solve linear systems of equations
- Numpy provides the functionality to calculate the determinant of a matrix.
- Numpy provides the functionality to calculate the inverse of a matrix.



# Lineal Algebra with Numpy

- **Numpy** provides a **comprehensive set** of **linear algebra** functions.
- **Numpy** provides the **functionality** to **create** and **manipulate matrices**.
- **Numpy** provides the **functionality** to **solve linear systems** of equations.

- **Numpy** provides the **functionality** to **calculate the determinant** of a matrix.
- **Numpy** provides the **functionality** to **calculate the inverse** of a matrix.

$$\begin{bmatrix} 2 & 1 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$



# Lineal Algebra with Numpy

- **Numpy** provides a **comprehensive set** of **linear algebra** functions.
- **Numpy** provides the **functionality** to **create** and **manipulate matrices**.
- **Numpy** provides the **functionality** to **solve linear systems** of equations.
- **Numpy** provides the **functionality** to **calculate** the **determinant** of a matrix.
- **Numpy** provides the **functionality** to **calculate** the **inverse** of a matrix.



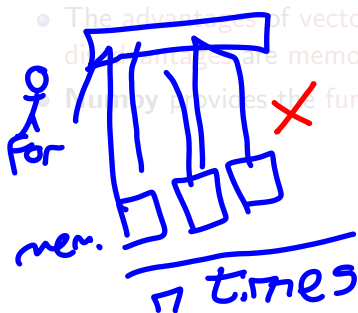
# Lineal Algebra with Numpy

- **Numpy** provides a **comprehensive set** of **linear algebra** functions.
- **Numpy** provides the **functionality** to **create** and **manipulate matrices**.
- **Numpy** provides the **functionality** to **solve linear systems** of equations.
- **Numpy** provides the **functionality** to **calculate** the **determinant** of a matrix.
- **Numpy** provides the **functionality** to **calculate** the **inverse** of a matrix.



# Vectorization with Numpy

- **Vectorization** is the process of converting an algorithm from operating on a single value at a time to operating on a set of values at one time.
- **Vectorization** is the process of replacing explicit loops with array expressions or matrix operations.



# Vectorization with Numpy

- **Vectorization** is the process of converting an algorithm from operating on a single value at a time to operating on a set of values at one time.
- **Vectorization** is the process of replacing explicit loops with array expressions or matrix operations.
- The advantages of vectorization are speed and clarity. The disadvantages are memory and complexity.
- **Numpy** provides the functionality to vectorize operations on arrays.



# Typical Operations with Numpy

- **Numpy** provides the functionality to create and manipulate arrays.
- **Numpy** provides the functionality to perform element-wise operations on arrays. matrix operations on arrays.
- **Numpy** provides the functionality to perform linear algebra operations on arrays.
- **Numpy** provides the functionality to perform statistical operations on arrays.

↓  
descriptive



# Outline

- 1 Foundations of Python
- 2 Numerical Analysis with Numpy
- 3 Data Manipulation with Pandas



# Introduction to Pandas

- 7p.
- **Pandas** is a fast, powerful, flexible, and easy-to-use open-source data manipulation and data analysis library built on top of the Python programming language.
  - **Pandas** is a high-level data manipulation tool developed by Wes McKinney in 2008.

• Pandas is a fast and efficient data manipulation tool that is built on top of NumPy.

• Pandas is one of the most popular and widely-used data manipulation libraries in the world.

Excel  
Python



# Introduction to Pandas

- **Pandas** is a fast, powerful, flexible, and easy-to-use open-source data manipulation and data analysis library built on top of the Python programming language.
- **Pandas** is a high-level data manipulation tool developed by Wes McKinney in 2008.
- **Pandas** is a fast and efficient data manipulation tool that is built on top of NumPy.
- **Pandas** is one of the most popular and widely-used data manipulation libraries in the world.



# The "Series" Data Structure

- A **Series** is a one-dimensional array-like object that contains a sequence of values and an associated array of data labels, called the index.
- The index of a **Series** is an array of labels that correspond to the values in the **Series**. The index of a **Series** is an optional parameter that defaults to a sequence of integers starting at zero.
- The **Series** object is a core data structure in **Pandas**.

Series

$a_0$   
 $a_1$   
 $a_2$   
 $\vdots$   
 $a_n$

index  
 "first" 0  
 "second" 1  
 "third" 2  
 $\vdots$   
 "n-th" n

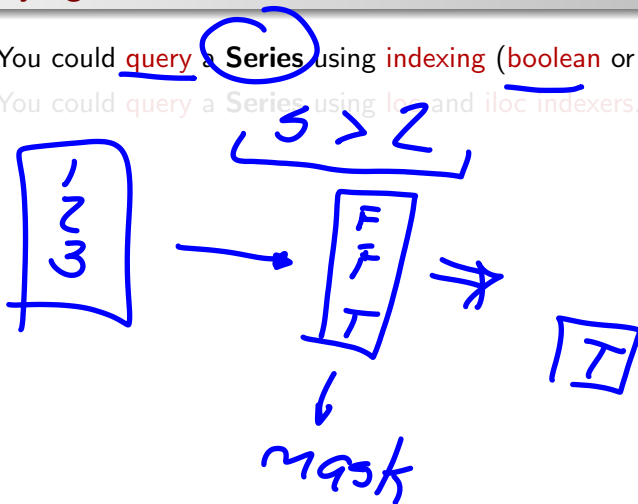
Array  
 index 0 1 2 ... n  
 $[a_0, a_1, \dots, a_n]$

NaN: Not a Number  
 Float



# Querying a Series

- You could query a Series using indexing (boolean or fancy).
- You could query a Series using loc and iloc indexers.



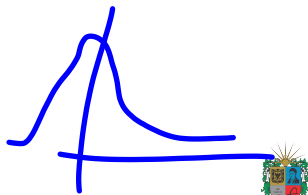
# Querying a Series

- You could **query** a **Series** using **indexing** (boolean or fancy).
- You could **query** a **Series** using **loc** and **iloc** indexers.

Handwritten table illustrating indexing:

|   | index | value |
|---|-------|-------|
| 0 | a     | 12    |
| 1 | b     | 24    |
| 2 | c     | 36    |

not visible  $\Rightarrow$  **iloc**



# The "DataFrame" Data Structure

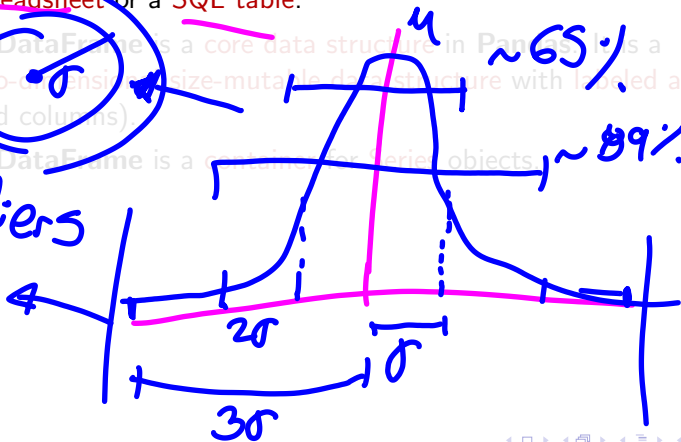
- A **DataFrame** is a two-dimensional labeled data structure with columns of potentially different types.

- A **DataFrame** is a tabular data structure that is similar to a spreadsheet or a SQL table.

- A **DataFrame** is a core data structure in Pandas. It is a two-dimensional size-mutable data structure with labeled axes (rows and columns).

- A **DataFrame** is a container for Series objects.

outliers



# The “DataFrame” Data Structure

- A **DataFrame** is a two-dimensional labeled data structure with columns of potentially different types.
- A **DataFrame** is a tabular data structure that is similar to a spreadsheet or a SQL table.
- A **DataFrame** is a core data structure in **Pandas**. It is a two-dimensional size-mutable data structure with labeled axes (rows and columns).
- A **DataFrame** is a container for Series objects.



# DataFrame Indexing and Loading

- You could **index** a **DataFrame** using **column names**.
- You could load a DataFrame from a CSV file.
- You could load a DataFrame from a JSON file.
- You could load a DataFrame from a SQL database.

② values



# DataFrame Indexing and Loading

- You could **index** a **DataFrame** using **column names**.
- You could **load** a **DataFrame** from a **CSV file**.
- You could **load** a **DataFrame** from a **JSON file**.
- You could **load** a **DataFrame** from a **SQL database**.



# DataFrame Indexing and Loading

- You could **index** a **DataFrame** using **column names**.
- You could **load** a **DataFrame** from a **CSV file**.
- You could **load** a **DataFrame** from a **JSON file**.
- You could **load** a **DataFrame** from a **SQL database**.



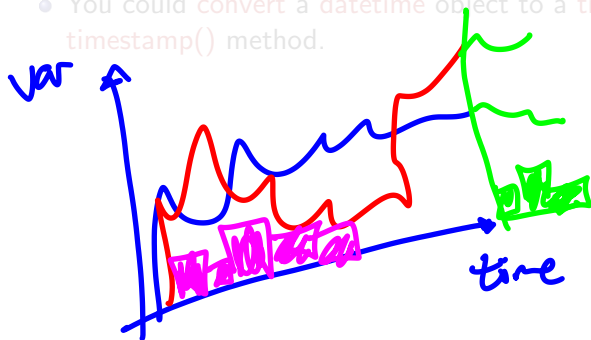
# DataFrame Indexing and Loading

- You could **index** a **DataFrame** using **column names**.
- You could **load** a **DataFrame** from a **CSV file**.
- You could **load** a **DataFrame** from a **JSON file**.
- You could **load** a **DataFrame** from a **SQL database**.



# DateTime Handling in Pandas

- You could **convert** a **string** to a **datetime** object using the `to_datetime()` method.
- You could **convert** a **datetime** object to a **string** using the `strftime()` method.
- You could **convert** a **datetime** object to a **timestamp** using the `timestamp()` method.



# DateTime Handling in Pandas

- You could **convert** a **string** to a **datetime** object using the **to\_datetime()** method.
- You could **convert** a **datetime** object to a **string** using the **strftime()** method.
- You could **convert** a **datetime** object to a **timestamp** using the **timestamp()** method.



# DateTime Handling in Pandas

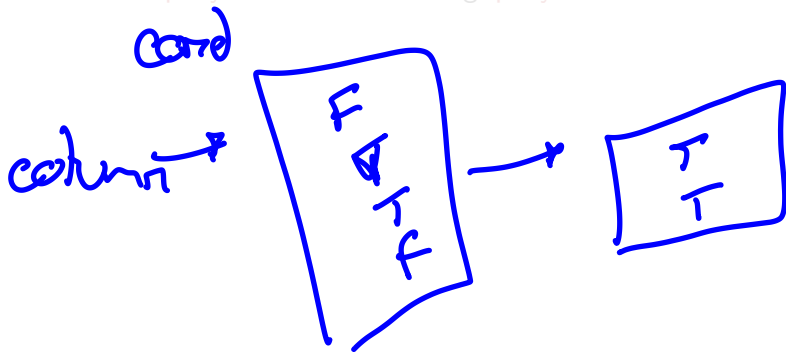
- You could **convert** a **string** to a **datetime** object using the **to\_datetime()** method.
- You could **convert** a **datetime** object to a **string** using the **strftime()** method.
- You could **convert** a **datetime** object to a **timestamp** using the **timestamp()** method.

seconds 1 Feb 1973



# Querying a DataFrame

- You could **query** a **DataFrame** using **indexing** (**boolean or fancy**).
- You could **query** a **DataFrame** using **loc** and **iloc** indexers.
- You could **query** a **DataFrame** using **query** method.



# Querying a DataFrame

- You could **query** a **DataFrame** using **indexing** (**boolean** or **fancy**).
- You could **query** a **DataFrame** using **loc** and **iloc** indexers.
- You could **query** a **DataFrame** using **query** method.

Handwritten notes in blue ink:

- A circle around **loc** with an arrow pointing to the word **column** written below it.
- A circle around **iloc** with an arrow pointing to the word **index** written below it.
- To the right of **index**, a vertical ellipsis (three dots) followed by a downward-pointing arrow.



# Querying a DataFrame

- You could **query** a **DataFrame** using **indexing** (**boolean** or **fancy**).
- You could **query** a **DataFrame** using **loc** and **iloc** indexers.
- You could **query** a **DataFrame** using **query method**.



# Missing Values in a DataFrame

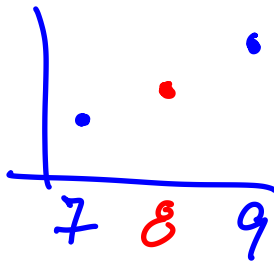
- You could **detect** missing values in a **DataFrame**. The **isnull()** method returns a **Boolean DataFrame** indicating the **presence** of missing values.
- You could **fill** missing values in a **DataFrame**. The **fillna()** method returns a **DataFrame** with missing values filled.
- You could **drop** missing values in a **DataFrame**. The **dropna()** method returns a **DataFrame** with missing values dropped.



# Missing Values in a DataFrame

- You could **detect** missing values in a **DataFrame**. The **isnull()** method returns a **Boolean DataFrame** indicating the **presence** of missing values.
- You could **fill** missing values in a **DataFrame**. The **fillna()** method returns a **DataFrame** with missing values filled.
- You could **drop** missing values in a **DataFrame**. The **dropna()** method returns a **DataFrame** with missing values dropped.

Imputers



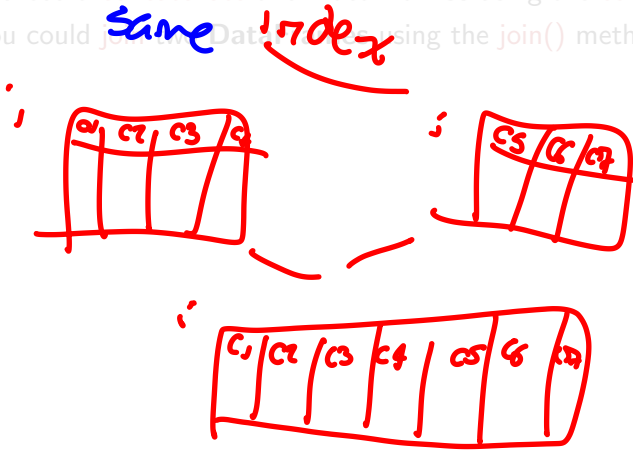
# Missing Values in a DataFrame

- You could **detect** missing values in a **DataFrame**. The **isnull()** method returns a **Boolean DataFrame** indicating the **presence** of missing values.
- You could **fill** missing values in a **DataFrame**. The **fillna()** method returns a **DataFrame** with missing values filled.
- You could **drop** missing values in a **DataFrame**. The **dropna()** method returns a **DataFrame** with missing values dropped.



# Merging DataFrames

- You could **merge** two **DataFrames** using the **merge()** method.
- You could concatenate two DataFrames using the **concat()** method.
- You could join two DataFrames using the **join()** method.



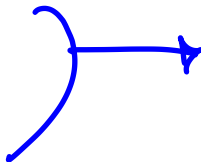
# Merging DataFrames

- You could **merge** two **DataFrames** using the **merge()** method.
- You could **concatenate** two **DataFrames** using the **concat()** method.
- You could **join** two **DataFrames** using the **join()** method.

↳ same columns

|    | c1 | c2 | c3 |
|----|----|----|----|
| 0  |    |    |    |
| 10 |    |    |    |

|    | c1 | c2 | c3 |
|----|----|----|----|
| 15 |    |    |    |
| 20 |    |    |    |



|    | c1 | c2 | c3 |
|----|----|----|----|
| 0  |    |    |    |
| 10 |    |    |    |
| 15 |    |    |    |
| 20 |    |    |    |



# Merging DataFrames

- You could **merge** two **DataFrames** using the **merge()** method.
- You could **concatenate** two **DataFrames** using the **concat()** method.
- You could **join** two **DataFrames** using the **join()** method.

SQL → any column  
inner  
left  
right  
outer



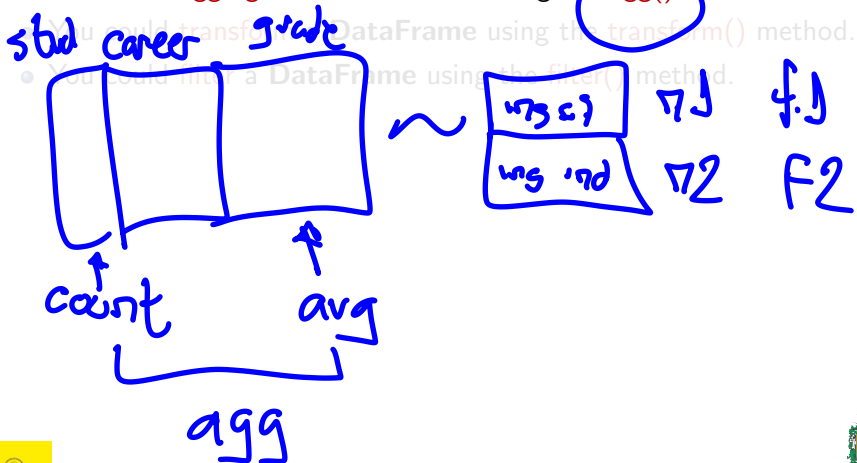
# GroupBy in Pandas

- You could **group** a **DataFrame** using the **groupby()** method.
- You could **aggregate** a **DataFrame** using the **agg()** method.
- You could **transform** a **DataFrame** using the **transform()** method.
- You could **filter** a **DataFrame** using the **filter()** method.



# GroupBy in Pandas

- You could **group** a **DataFrame** using the **groupby()** method.
- You could **aggregate** a **DataFrame** using the **agg()** method.



# GroupBy in Pandas

- You could **group** a **DataFrame** using the **groupby()** method.
- You could **aggregate** a **DataFrame** using the **agg()** method.
- You could **transform** a **DataFrame** using the **transform()** method.
- You could **filter** a **DataFrame** using the **filter()** method.



# GroupBy in Pandas

- You could **group** a **DataFrame** using the **groupby()** method.
- You could **aggregate** a **DataFrame** using the **agg()** method.
- You could **transform** a **DataFrame** using the **transform()** method.
- You could **filter** a **DataFrame** using the **filter()** method.



# Pandas Idioms

- You could **apply functions** to **DataFrames**.
- You could chain methods in Pandas.

*•apply (lambda q...)*



## Pandas Idioms

- You could **apply functions** to **DataFrames**.
- You could **chain methods** in **Pandas**.

df.open1().open2().open3()



# Outline

- 1 Foundations of Python
- 2 Numerical Analysis with Numpy
- 3 Data Manipulation with Pandas



# Thanks!

## Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

